République Tunisienne
Le ministère de l'Enseignement supérieur et de la Recherche scientifique
L'université de Tunis El Manar
École Nationale des Ingénieurs de Tunis

Département Génie Électrique

Rapport de Stage Ingénieur

Conception et réalisation d'un nœud de capteurs en communication avec un serveur MQTT

Effectué par :
Nerimen SAHLI

Classe
2AGE1

Effectué à :

Année Universitaire : 2025–2026

Remerciements

Je tiens à exprimer ma profonde gratitude envers toutes les personnes qui ont contribué au bon déroulement de ce stage.

Mes remerciements s'adressent tout particulièrement à mon encadrant, Monsieur Hichem Habchi, pour sa confiance, son aide précieuse ainsi que ses conseils enrichissants, tant sur le plan technique que personnel. Je remercie également l'ensemble de l'équipe de CCNS pour son accueil chaleureux et sa bienveillance.

Enfin, j'adresse mes sincères remerciements aux professeurs de l'École Nationale d'Ingénieurs de Tunis (ENIT) dont la qualité de la formation m'a apporté les fondements scientifiques et techniques nécessaires à la réalisation du projet au cadre de ce stage.

Table des matières

Table des figures **4**

Liste des tableaux **5**

Liste des abréviations **6**

Introduction générale **7**

1 Contexte Général, Problématique et Recherche Bibliographique **8**
1.1 Introduction 8
1.2 Présentation de l'organisme d'accueil 8
1.2.1 Présentation générale 8
1.2.2 Positionnement sur le marché 8
1.2.3 Organisation interne 9
1.2.4 Cadre du stage 9
1.3 Contexte et Problématique 9
1.3.1 Contexte du projet 9
1.3.2 Problématique 10
1.3.3 Objectifs du travail 10
1.4 Étude Bibliographique 11
1.4.1 Les systèmes de monitoring des Data Centers 11
1.4.2 Architecture de base : Choix du microcontrôleur et du protocole 12
1.4.2.1 Choix du microcontrôleur 12
1.4.2.2 Choix du protocole de communication 13
1.4.3 Choix des composants d'acquisition 13
1.5 Conclusion 15

2 Architecture Matérielle et Logicielle du Système **17**
2.1 Introduction 17
2.2 Conception Électrique 17
2.2.1 Schéma de câblage 17
2.2.2 Gestion de l'alimentation et adaptation des signaux 18
2.2.3 Réalisation du prototype matériel 18
2.3 Architecture Logicielle 20
2.3.1 Choix architectural : L'approche non-bloquante des FSM 20
2.3.2 Modélisation FSM de la résilience réseau 21
2.3.3 Modélisation FSM de l'acquisition et de la gestion des alertes 22
2.4 Développement et Programmation 23
2.4.1 Lecture des capteurs 23

2.4.2 Vérification des alertes . . . 23
2.4.3 Publication MQTT : Mesures et Alertes . . . 24
2.4.4 Basculement réseau automatisé . . . 24
2.4.5 Transmission des SMS d'urgence . . . 24
2.5 Intégration d'une Interface Locale . . . 24
2.5.1 Câblage et intégration matérielle . . . 25
2.5.2 Gestion de l'affichage . . . 25
2.5.3 Intégration logicielle et gestion de l'affichage . . . 26
2.6 Tests et Résultats . . . 26
2.6.1 Initialisation et connectivité Wi-Fi . . . 26
2.6.2 Supervision électrique et alertes SMS . . . 27
2.6.3 Validation des capteurs environnementaux et de sécurité . . . 28
2.6.4 Superposition et indépendance des alertes . . . 30
2.6.5 Mécanisme de basculement réseau . . . 31
2.6.6 Validation de l'affichage sur l'écran OLED . . . 31
2.7 Analyse de l'Empreinte Mémoire . . . 32
2.8 Bilan Financier et Estimation des Coûts . . . 32
2.8.1 Nomenclature des coûts du prototype . . . 32
2.8.2 Analyse comparative avec les solutions du marché . . . 33
2.8.3 Synthèse et Gain Économique . . . 33
2.9 Conclusion et Perspectives d'Évolution . . . 33

Conclusion générale **35**
Bibliographie et Webographie . . . 36

Bibliographie et Webographie **36**

A Extraits du code **38**
A.1 Code de lecture des capteurs . . . 38
A.2 Code de vérification des alertes . . . 39
A.3 Code de publication des mesures . . . 39
A.4 Code de publication des alertes . . . 40
A.5 Extrait de la FSM réseau . . . 41
A.6 Fonction de l'envoi d'SMS . . . 42
A.7 Fonction d'affichage sur l'écran . . . 43

Table des figures

1.1 L'organigramme général de CCNS 9
1.2 Carte LilyGO TTGO T-Call 12
1.3 L'ensemble des modules d'acquisition et capteurs utilisés dans le projet 15

2.1 Schéma de câblage global du système 18
2.2 Abaisseur de tension 18
2.3 Prototype physique du nœud capteur assemblé sur plaque d'essai 19
2.4 Architecture globale du système 20
2.5 Diagramme d'états de l'FSM de résilience réseau 22
2.6 Diagramme d'états de l'automate d'acquisition et de gestion des alertes 23
2.7 Schéma de câblage incluant l'écran OLED 25
2.8 Simulation de l'affichage OLED avec les icônes d'alerte actives 26
2.9 Connexion établie au réseau Wi-Fi 27
2.10 SMS d'alerte reçu lors de la coupure de courant secteur 27
2.11 Mesures publiées lors de la coupure de courant 28
2.12 Mesures publiées après le rebranchement de la charge 28
2.13 Message de fin d'alerte coupure de courant 28
2.14 Résultats des tests effectuées pour les cas de fuite d'eau, de vibration et de surchauffe 29
2.15 Variation de la température autour du seuil prédéfini 29
2.16 Augmentation de l'humidité jusqu'au seuil prédéfini 30
2.17 Publication de l'alerte en cas d'humidité élevée 30
2.18 Résultats des tests effectuées pour le cas de détection d'une flamme 30
2.19 Résultat du test avec un scénario de pannes simultanées 31
2.20 SMS d'alerte reçu lors de la coupure de WiFi 31
2.21 L'écran OLED affichant la télémétrie, le statut réseau et des icônes d'alerte . . 31

Liste des tableaux

1.1 Comparaison des plateformes de traitement matérielles 12

2.1 Bilan financier de la réalisation du prototype 32
2.2 Comparaison financière entre le prototype et les solutions du marché 33

Liste des abréviations

ADC	Analog-to-Digital Converter
ASHRAE	American Society of Heating, Refrigerating and Air-Conditioning Engineers
CCNS	Company of Computing and Network Services
CT	Current Transformer (Transformateur de Courant)
DT / TND	Dinar Tunisien
ENIT	École Nationale d'Ingénieurs de Tunis
FSM	Finite State Machine (Machine à États Finis)
GPIO	General Purpose Input/Output
GPRS	General Packet Radio Service
GSM	Global System for Mobile Communications
HTTP	Hypertext Transfer Protocol
I2C	Inter-Integrated Circuit
IHM	Interface Homme-Machine
IoT	Internet of Things
IP	Internet Protocol
JSON	JavaScript Object Notation
MQTT	Message Queuing Telemetry Transport
NO	Normalement Ouvert
OLED	Organic Light-Emitting Diode
OS	Operating System
PCB	Printed Circuit Board
PF	Power Factor (Facteur de Puissance)
SCL	Serial Clock
SDA	Serial Data
SMS	Short Message Service
SRAM	Static Random-Access Memory
UART	Universal Asynchronous Receiver-Transmitter

Introduction générale

À l'ère du numérique et de la transformation digitale, la continuité du service des infrastructures informatiques est un enjeu critique pour les entreprises. Les Data Centers et salles serveurs qui hébergent les équipements indispensables dépendent de la maintenance logicielle et réseau, mais aussi du maintien de conditions environnementales et électriques précises. Tout problème peut entraîner des pannes matérielles et des pertes financières importantes. Bien que le marché propose plusieurs solutions de supervision, elles sont généralement propriétaires, rigides et coûteuses.

C'est dans ce cadre que s'inscrit ce projet de stage d'ingénieur effectué au sein de l'entreprise Company of Computing and Network Services (CCNS) pour la période de deux mois.

L'objectif principal de ce projet est de concevoir et réaliser un nœud de capteurs IoT économique et résilient, qui soit capable d'assurer la mesure des paramètres physiques et énergétiques, de transmettre ces mesures et d'alerter les utilisateurs en cas de problème.

Ce rapport retrace les étapes de réalisation du projet dans deux chapitres principaux :

— Le Chapitre 1 présente le contexte général du projet, la problématique et l'étude bibliographique.
— Le Chapitre 2 présente l'architecture matérielle et logicielle du système, la conception électrique, l'approche par machines à états finis, la tolérance aux pannes réseau, la transmission des SMS d'urgence, l'intégration d'une interface graphique locale et les tests effectués sur le prototype.

Enfin, ce rapport se termine par le bilan général, les perspectives d'évolution et la conclusion.

Chapitre 1

Contexte Général, Problématique et Recherche Bibliographique

1.1 Introduction

Ce premier chapitre pose le cadre général du projet effectué au sein de l'entreprise CCNS. Il commence par la présentation de l'entreprise d'accueil, ses domaines d'expertise, son positionnement sur le marché et son organisation interne. Ensuite, il définit le contexte et la problématique à résoudre et les objectifs fixés dans le cahier des charges. Enfin, il présente l'étude bibliographique menée afin de faire les choix du microcontrôleur, du protocole de communication et des capteurs.

1.2 Présentation de l'organisme d'accueil

Cette section présente l'entreprise CCNS, son positionnement sur le marché ainsi que son organisation interne, avant de préciser le cadre spécifique dans lequel s'est déroulé le stage.

1.2.1 Présentation générale

Depuis sa fondation en 2011, CCNS est une entreprise privée spécialisée dans les solutions de technologies de l'information qui opère dans des domaines divers pour mieux répondre aux exigences complexes de ses clients. Proche des problématiques métiers, l'entreprise a acquis une expérience solide et a développé une expertise pointue dans des domaines tels que :

— L'implémentation des infrastructures réseau et des Systèmes d'Information.

— Le développement de solutions Web sur mesure.

— L'informatique décisionnelle (Business Intelligence).

— L'hébergement et le Cloud [1].

1.2.2 Positionnement sur le marché

La société opère non seulement sur le marché des entreprises en Tunisie mais aussi en France, en Allemagne et en Afrique. En effet, elle a déjà réalisé de nombreux projets et possède un important portefeuille de clients dont Landor, Vitalait et Instadeep. Elle possède aussi une liste de partenaires dont Microsoft, Cisco, IBM, DELL et VMWARE [1].

1.2.3 Organisation interne

Pour s'assurer du bon déroulement des projets face aux exigences des clients, la CCNS a une organisation interne bien structurée. L'entreprise se divise en plusieurs pôles qui combinent l'ingénierie informatique, les télécommunications, l'analyse de données et la gestion commerciale. La Figure 1.1 illustre l'organigramme général de l'entreprise.

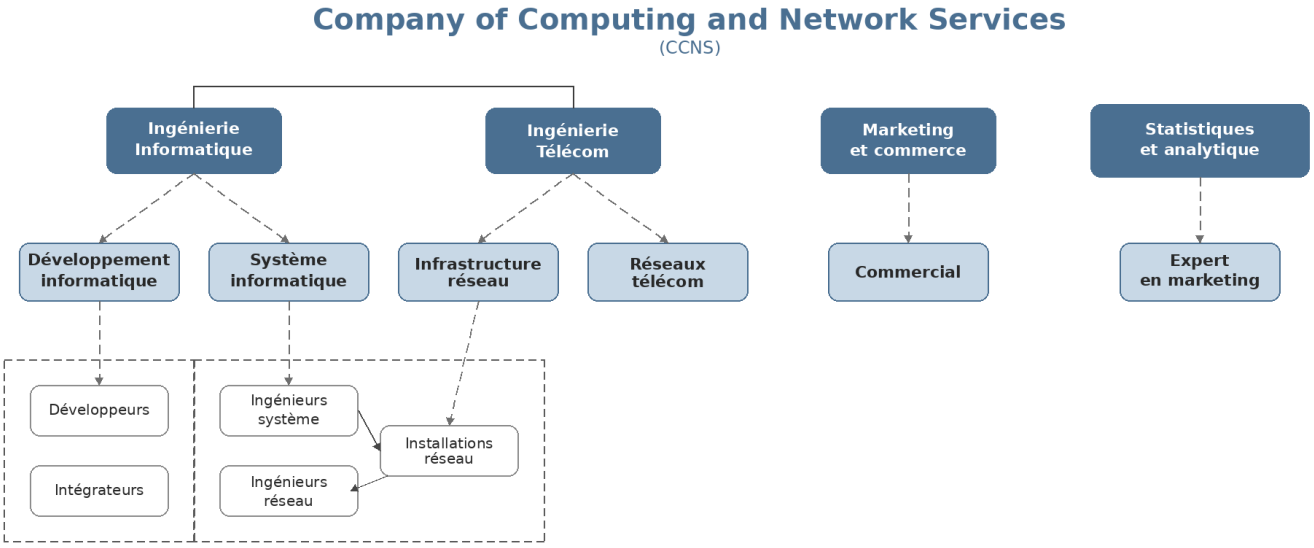


FIGURE 1.1 – L'organigramme général de CCNS

1.2.4 Cadre du stage

Bien que la CCNS soit spécialisée en ingénierie logicielle, l'évolution vers l'IoT exige aujourd'hui une intégration avec les capteurs et équipements physiques. Mon rôle au sein de l'équipe a consisté à répondre à ce besoin, en agissant comme trait d'union entre la conception matérielle et les solutions logicielles de l'entreprise.

1.3 Contexte et Problématique

Cette section situe le projet dans son contexte, formule la problématique à résoudre et détaille les objectifs fixés par le cahier des charges.

1.3.1 Contexte du projet

De nos jours, la gestion des salles serveurs se limite non seulement à la maintenance logicielle et réseau, mais aussi aux conditions environnementales comme la température et l'humidité, et à la consommation énergétique. Ces paramètres jouent un rôle important dans la durabilité et les performances du matériel. Les entreprises cherchent à éviter les dégâts et pertes coûteuses et à maximiser l'efficacité [2].

Dans ce contexte est née l'idée de ce projet au sein de la CCNS. L'entreprise a identifié le besoin d'un système de surveillance automatisé et en temps réel pour optimiser le suivi de ses infrastructures et de celles de ses clients.

1.3.2 Problématique

Bien que l'objectif global de mettre en place ce système de surveillance soit bien défini, la problématique de ce projet se résume à la question suivante :

Comment concevoir un système embarqué IoT fiable qui soit capable de mesurer les conditions environnementales et énergétiques, tout en s'interfaçant avec les plateformes logicielles de la CCNS ?

Pour répondre à cette problématique, plusieurs défis techniques doivent être relevés lors de ce projet :

— Comment choisir et intégrer plusieurs capteurs autour d'un microcontrôleur central ?

— Comment garantir une transmission de données et d'alertes continue vers le serveur de l'entreprise ?

— Comment créer une passerelle entre l'environnement physique des salles serveurs et l'interface web développée par les équipes de la CCNS ?

1.3.3 Objectifs du travail

Afin de répondre à cette problématique, l'objectif principal de ce projet est la conception et réalisation d'un nœud de capteurs en communication avec un serveur MQTT. Ce système doit satisfaire un cahier des charges qui se divise en trois axes majeurs :

— **Acquisition continue des données :** Mesurer en temps réel les paramètres énergétiques (tension, courant et facteur de puissance) et environnementaux (température et humidité), puis transmettre ces mesures via le protocole MQTT.

— **Détection et gestion des alertes :** Surveiller l'infrastructure physique pour identifier instantanément les anomalies critiques. Le système doit détecter les fuites d'eau ou de gaz, la présence de flammes, les vibrations, l'ouverture d'une porte, les coupures de courant et tout dépassement de seuil toléré pour les grandeurs électriques et environnementales mesurées. Le nœud doit publier des messages d'alerte lors du déclenchement d'un événement, et des messages de fin d'alerte lors du retour à la normale, et ce via le protocole MQTT.

— **Basculement automatique entre connexion WiFi et cellulaire :** Pour garantir une communication continue même en cas de défaillance de l'infrastructure, le système doit basculer automatiquement vers une connexion cellulaire en cas de perte du signal Wi-Fi. De plus, il doit alerter directement l'administrateur par l'envoi d'un SMS d'urgence lors d'une coupure de courant ou d'une déconnexion Wi-Fi. Le système doit rebasculer automatiquement vers la connexion WiFi dès que possible.

Enfin, pour que cette solution réponde parfaitement aux besoins de l'entreprise, le système doit être robuste, de faible latence de communication et d'un coût de réalisation optimisé.

1.4 Étude Bibliographique

Cette section présente la recherche bibliographique menée afin de justifier les choix techniques de microcontrôleur, de protocole de communication et des capteurs d'acquisition.

1.4.1 Les systèmes de monitoring des Data Centers

Les data centers constituent aujourd'hui un pilier fondamental des infrastructures technologiques. Qu'ils prennent la forme de salles dédiées au sein d'entreprises de taille moyenne, de gigantesques fermes de serveurs ou d'infrastructures de cloud computing, la sécurité matérielle des équipements, particulièrement des serveurs informatiques, demeure une priorité absolue. Parmi les importants facteurs à surveiller sont la température, l'humidité et la disponibilité de l'alimentation électrique et du signal WiFi. Par exemple, La norme américaine ASHRAE recommande une température entre 18°C et 27°C pour les équipements informatiques, et une humidité relative entre 20% et 80%. Ces paramètres ont un effet direct sur la durée de vie des composants électroniques [2].

Il est donc important d'équiper ces centres de systèmes de surveillance qui garantissent, en permanence, le maintien des bonnes conditions environnementales et énergétiques. Sur le marché, plusieurs solutions se présentent :

— **NetBotz par APC (Schneider Electric) :** Ces appareils protégent les infrastructures informatiques contre les menaces environnementales en détectant des risques tels que l'humidité extrême, les inondations, les incendies ou la fumée. Une interface utilisateur accessible via un navigateur web permet de visualiser les données des capteurs et le journal des alertes [3].

— **Room Alert par AVTECH :** Il s'agit de systèmes de surveillance environnementale dédiés à la protection des équipements informatiques et des centres de données. Ces dispositifs mesurent la température, l'humidité, l'état de l'alimentation, les fuites d'eau, la fumée et le flux d'air. Ils génèrent également des notifications par e-mail ou SMS et permettent une gestion à distance via un tableau de bord ou un logiciel dédié [4].

— **Smart Rack Controller par Raritan :** Cette solution agit comme un point de connexion central qui permet d'intégrer des dizaines de capteurs intelligents dédiés à la surveillance des conditions environnementales. Elle offre la possibilité de définir des seuils critiques et de recevoir des alertes. L'ensemble est controlable via une interface web ou une application mobile [5].

Bien que ces solutions soient fiables et répandues sur le marché, elles présentent des limitations. Elles sont coûteuses, peu personnalisables et s'inscrivent dans des écosystèmes propriétaires fermés. Face à ces contraintes techniques et financières, la réalisation de ce projet permet de se passer de ces systèmes propriétaires. Elle offre une personnalisation infinie quant au choix des composants, permet de réduire les coûts de production, et garantit le controle total sur l'intégration et le traitement des données.

1.4.2 Architecture de base : Choix du microcontrôleur et du protocole

La première étape consiste à définir l'architecture de base du système, en sélectionnant l'unité de traitement et le standard de communication adaptés au cahier des charges.

1.4.2.1 Choix du microcontrôleur

Le microcontrôleur doit offrir une puissance de calcul suffisante pour gérer l'acquisition de multiples capteurs analogiques et numériques et assurer la transmission des données de manière continue. En plus, il doit être résilient face aux pannes électriques. Afin de faire un choix judicieux, il faut comparer les microcontrôleurs les plus utilisés dans le domaine de l'IoT : la famille Arduino, les nano-ordinateurs Raspberry Pi, et l'ESP32 d'Espressif. Le tableau suivant 1.1 illustre leurs points forts et limites respectifs pour la tâche en question :

Critère	Arduino	Raspberry Pi 4	ESP32
Connectivité réseau	Nécessite des modules externes	Wi-Fi, Bluetooth, Ethernet	Native Wi-Fi, Bluetooth
Puissance et Multi-tâche	Faible (Architecture mono-cœur simple)	Très élevée (Microprocesseur et OS Linux)	Élevée (Double cœur)
Consommation énergétique	Faible	Élevée	Très faible
Tolérance aux coupures de courant	Redémarrage instantané	Risque de corruption de la carte SD	Redémarrage instantané
Coût d'acquisition	Moyen	Très élevé	Très faible

TABLE 1.1 – Comparaison des plateformes de traitement matérielles

L'ESP32 offre un parfait compromis entre un coût minime, une connectivité Wi-Fi native indispensable à ce projet, et une architecture double cœur capable de gérer en continu la lecture des capteurs et l'envoi des requêtes réseau.

La carte de développement choisie est la **LilyGO TTGO T-Call** (Figure 1.2). Cette carte présente l'avantage d'intégrer sur un même circuit imprimé le microcontrôleur **ESP32-WROVER-B** et un module de communication cellulaire **SIM800L** compatible avec des Nano-SIM. Cette carte répond de manière optimale aux exigences. Elle permet de gérer nativement le basculement automatique vers le réseau mobile en cas de défaillance du Wi-Fi, et l'envoi des SMS, le tout sans câblage externe entre le microcontrôleur et le modem [8].

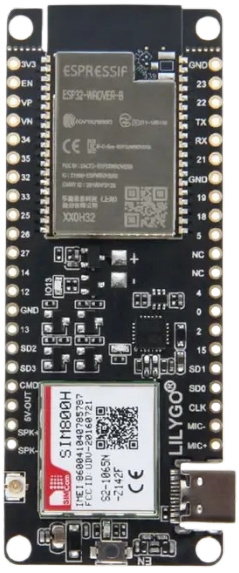


FIGURE 1.2 – Carte LilyGO TTGO T-Call

1.4.2.2 Choix du protocole de communication

Assurer la transmission continue des mesures et alertes entre l'ESP32 et l'infrastructure logicielle nécessite d'obéir aux contraintes suivantes :

— Les paquets de données à transmettre doivent être légers, car optimiser la bande passante est crucial lorsqu'on bascule sur le mode cellulaire où celle-ci est limitée.
— La transmission doit être en temps réel.
— La transmission doit rester stable même lorsque la connexion ne l'est pas.

Le protocole **HTTP** (Hypertext Transfer Protocol), bien qu'il soit le plus utilisé sur le web, est d'architecture de type "Client-Serveur" ou "requête-réponse", et ses en-têtes sont volumineuses, ajoutant 200 à 800 octets par message, ce qui le rend très lourd. Il utilise de nouvelles connexions pour chaque requête, entraînant des temps de réponse de 200 à 300 ms [6].

Par contre, le protocole **MQTT** (Message Queuing Telemetry Transport) est populaire en IoT. MQTT repose sur le modèle "Publication/Abonnement" (*Publish/Subscribe*) orchestré par un serveur centralisé appelé *Broker*. Son en-tête ne pèsant que 2 octets, il est d'une légèreté importante. Contrairement au protocole HTTP, il maintient une connexion persistante avec une moyenne de 43 ms pour 100 messages. Le nœud de capteurs se contente de publier les mesures et alertes sur des *Topics* spécifiques, et les applications logicielles de la CCNS n'ont qu'à s'abonner à ces Topics pour recevoir les données [6][7].

L'association de l'ESP32 et du protocole MQTT permet de construire une base matérielle et logicielle bien adaptée aux exigences du cahier des charges.

1.4.3 Choix des composants d'acquisition

Afin de concevoir le nœud de capteurs, le choix des composants s'est appuyé sur une étude comparative. Les capteurs suivants ont été choisis pour répondre aux exigences du cahier des charges.

— **Capteur de température et d'humidité (DHT22) (Figure 1.3a) :**
  — Le capteur DHT22 a été préféré au modèle DHT11, bien qu'il soit plus cher (16 TND contre 6,7 TND).
  — Il présente une marge d'erreur plus faible pour la mesure de la température ($\pm 0.5°C$ contre $\pm 2°C$) et de l'humidité ($\pm 2\%$ contre $\pm 5\%$).
  — Il offre une meilleure résolution, allant jusqu'à à $0.1°C$ et 0.1%.
  — Son alimentation requiert une tension de 3.3V à 5V [11].

— **Détecteur de fuites d'eau (SEN0454) (Figure 1.3b) :**
  — Le module SEN0454 de la marque DFRobot a été choisi pour détecter la présence d'eau.
  — Il nécessite une tension de fonctionnement comprise entre 7V et 18V.
  — Configuré en mode Normalement Ouvert (NO), les sondes du capteur constituent un circuit ouvert qui se ferme en présence d'eau (l'eau étant conductrice), retournant ainsi une tension vers le microcontrôleur [10].

— **Module de mesures électriques (PZEM-004T) (Figure 1.3c) :**
  — Pour la surveillance énergétique, le choix s'est porté sur le capteur PZEM-004T couplé à un transformateur de courant ouvrant (Split CT).
  — Ce système est non-intrusif, il est possible de l'installer sans avoir à couper le courant électrique.
  — Il est capable de mesurer une tension allant jusqu'à 260V sur une seule phase.
  — Il permet de mesurer un courant allant jusqu'à 100A, ainsi que le facteur de puissance sur une plage de 0 à 1.
  — Le module communique avec la carte ESP32 via une interface UART à un *baud rate* de 9600 [9].

— **Capteur de vibrations digital (SW-420) (Figure 1.3d) :**
  — Ce module a été intégré pour détecter les chocs, les mouvements anormaux ou les vibrations de l'équipement.
  — Il intègre un comparateur qui fournit un signal numérique.
  — Sa sensibilité aux vibrations est ajustable grâce à un potentiomètre intégré.
  — Il nécessite une tension d'alimentation comprise entre 3.3V et 5V [15].

— **Détecteur de porte ouverte (Capteur magnétique MC-38) (Figure 1.3e) :**
  — Il se compose de deux parties indépendantes : un aimant fixé sur la porte et le commutateur sur le cadre fixe.
  — Il fonctionne comme un interrupteur dont le circuit s'ouvre ou se ferme lorsque l'aimant s'éloigne, déclenchant ainsi une alerte [14].

— **Détecteur de gaz (MQ-135) (Figure 1.3f) :**
  — Ce capteur a été choisi pour prévenir les risques d'incendie ou de fuite en détectant la présence de gaz combustibles et de fumée.
  — Il requiert une tension d'alimentation de 5V indispensable pour faire fonctionner sa résistance chauffante interne.
  — Il offre une double interface : une sortie analogique proportionnelle à la concentration de gaz dans l'air, et une sortie numérique avec un seuil d'alerte réglable manuellement [12].

— **Détecteur de flamme (KY-026) (Figure 1.3g) :**
  — Ce module détecte les rayonnements infrarouges (longueur d'onde comprise entre 760 nm et 1100 nm) émis par une flamme.
  — Il possède un angle de détection d'environ 60 degrés.
  — Il intègre un comparateur LM393 qui délivre à la fois un signal analogique avec l'intensité de la flamme et un signal numérique d'alerte avec seuil ajustable via un potentiomètre.
  — Sa tension de fonctionnement se situe entre 3.3V et 5.5V [13].

La Figure 1.3 illustre l'ensemble des composants matériels sélectionnés pour l'acquisition des données.



(a) DHT22



(b) SEN0454



(c) PZEM-004T



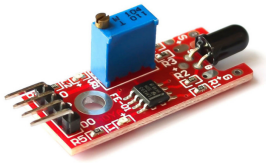
(d) SW-420



(e) MC-38



(f) MQ-135



(g) KY-026

FIGURE 1.3 – L'ensemble des modules d'acquisition et capteurs utilisés dans le projet

La combinaison de ces composants forme un nœud d'acquisition polyvalent assurant à la fois la surveillance énergétique et la sécurité matérielle de l'installation.

1.5 Conclusion

Ce chapitre a présenté le cadre du projet et l'étude bibliographique qui a guidé les choix techniques. L'étude de marché a souligné l'intérêt de développer une solution IoT personnalisée. Le choix de la carte LilyGO TTGO T-Call et du protocole MQTT, offre un compromis entre puissance, résilience et coût faible. De plus, les capteurs choisis permettent de couvrir tous les

risques ciblés. Ces choix théoriques établis, le chapitre suivant sera consacré à la conception matérielle, au développement de l'architecture logicielle, ainsi qu'à la validation expérimentale du prototype.

Chapitre 2

Architecture Matérielle et Logicielle du Système

2.1 Introduction

Après avoir posé les bases et choisi les composants adaptés, ce deuxième chapitre est consacré à la mise en œuvre pratique du nœud d'acquisition. Il présente la conception électrique du prototype, l'architecture logicielle basée sur deux machines à états finis (FSM) qui assurent l'acquisition des capteurs, la résilience de la connexion réseau, la publication MQTT et l'envoi des SMS d'urgence, ensuite l'intégration d'une interface graphique locale et enfin les résultats des tests menés sur l'ensemble du système.

2.2 Conception Électrique

Cette section décrit la conception électrique du prototype, du schéma de câblage jusqu'à la réalisation physique du montage.

2.2.1 Schéma de câblage

L'interfaçage entre la carte de traitement et les différents capteurs a été conçu comme illustré sur la Figure 2.1.

Il est à souligner que, suite à des contraintes d'approvisionnement lors de la phase de prototypage, des adaptations matérielles ont été réalisées afin de ne pas bloquer le développement logiciel et la validation du système :

— Le capteur DHT22 a été temporairement remplacé par un DHT11. Cette modification ne nécessite qu'un changement mineur au niveau du code.

— Les capteurs de vibration (SW-420) et de fuite d'eau (SEN0454) n'étant pas disponibles à temps, ils ont été simulés physiquement à l'aide de boutons-poussoirs. Cette méthode a permis de déclencher manuellement les alertes de vibration et de fuite d'eau.

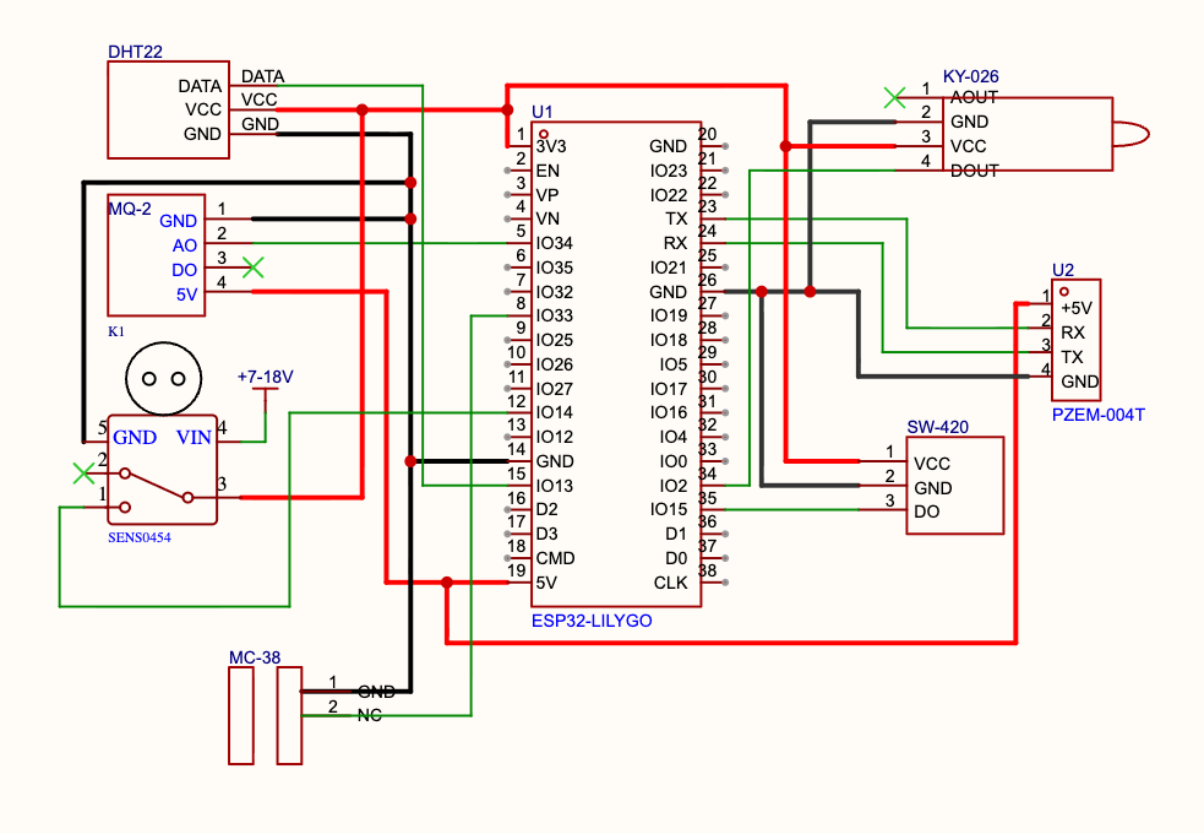


FIGURE 2.1 – Schéma de câblage global du système

2.2.2 Gestion de l'alimentation et adaptation des signaux

Le premier défi a été de combiner le capteur de gaz avec la carte, car il nécessite une tension d'alimentation de 5V pour fonctionner correctement.
Bien que la carte ESP32 soit capable de fournir cette tension, cela pose un problème de compatibilité pour les signaux de données. Les broches d'entrée analogique (ADC) de l'ESP32 ne tolèrent pas le 5V, et l'entrée GPIO reliée au capteur risque d'être détruite.

Pour protéger le microcontrôleur, un **abaisseur de tension** (Figure 2.2) a été intégré entre la sortie analogique du capteur de gaz et l'entrée de l'ESP32. Cette solution a permis de ramener l'amplitude maximale du signal de 5V à une plage sécurisée de 0 à 3,3V, garantissant ainsi une lecture en toute sécurité.

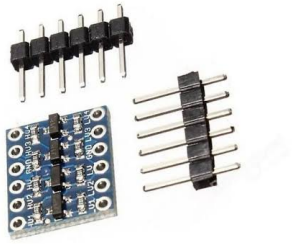


FIGURE 2.2 – Abaisseur de tension

2.2.3 Réalisation du prototype matériel

La Figure 2.3 présente la vue globale du montage réalisé et assemblé sur plaque d'essai. Ce montage fonctionnel a permis d'effectuer les tests de validation, d'ajuster les seuils de détection et de vérifier le basculement automatique entre les réseaux Wi-Fi et GSM.

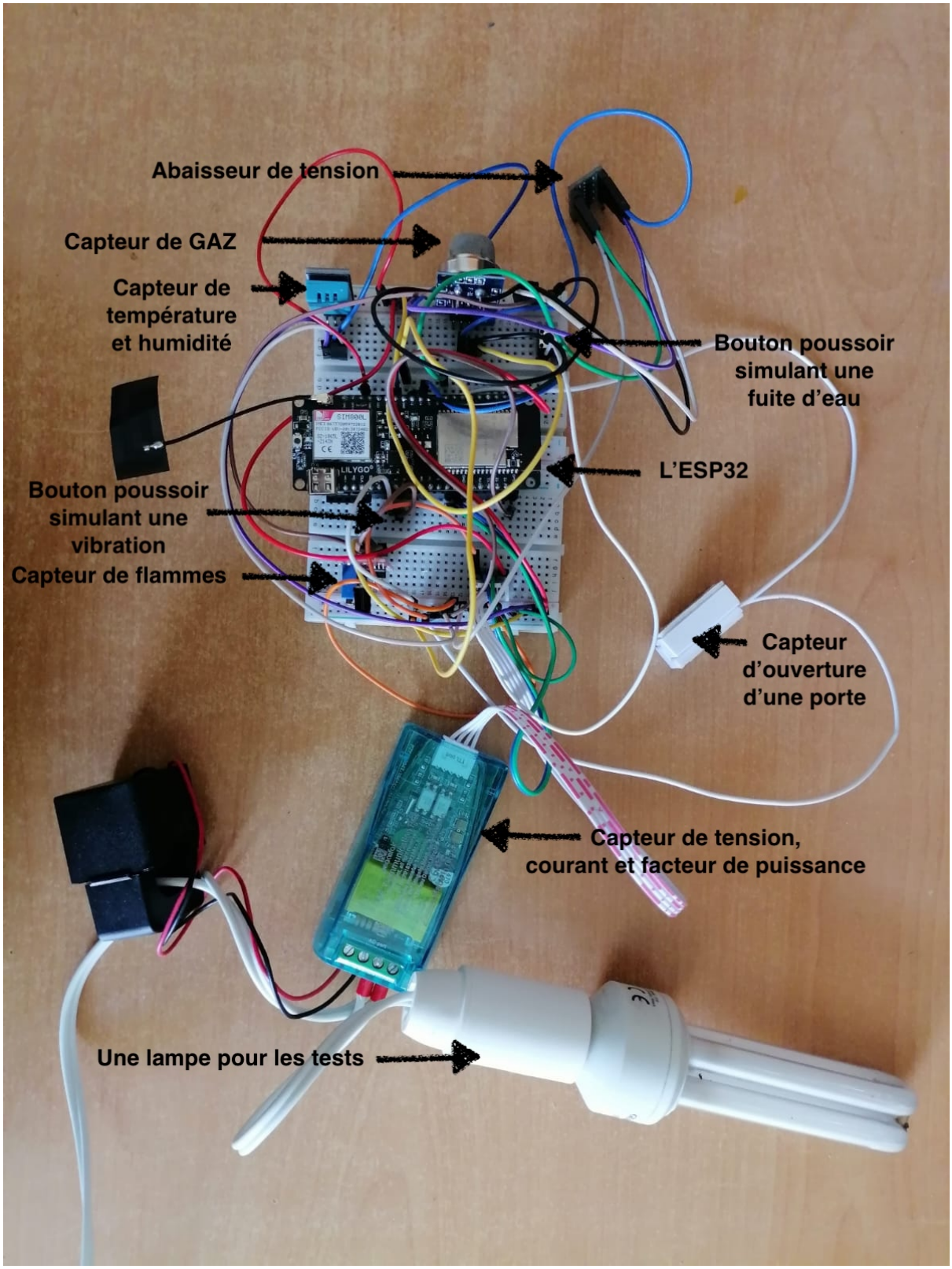


FIGURE 2.3 – Prototype physique du nœud capteur assemblé sur plaque d'essai

2.3 Architecture Logicielle

Avant de détailler l'architecture logicielle, la Figure 2.4 présente une vue d'ensemble du système dans sa globalité.
Le nœud de capteurs transmet ses données à l'ESP32, qui assure à la fois l'affichage local sur l'écran OLED et la transmission des données vers le broker MQTT, via une connexion Wi-Fi principale ou, en cas de défaillance de celle-ci, via une connexion cellulaire de secours qui déclenche également l'envoi de SMS d'alerte.

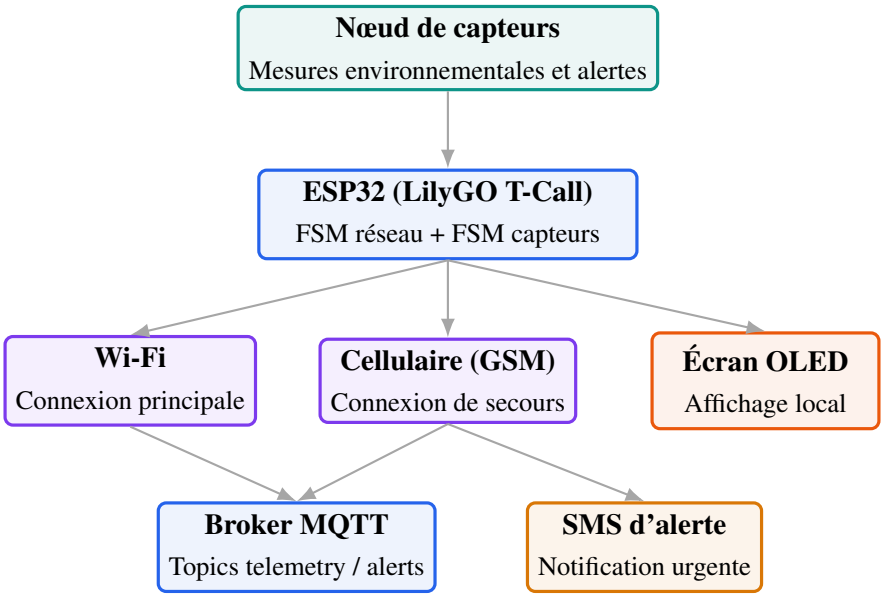


FIGURE 2.4 – Architecture globale du système

2.3.1 Choix architectural : L'approche non-bloquante des FSM

Lors de la programmation du microcontrôleur, il s'est vite avéré que l'utilisation de la fonction de pause *delay* et de la boucle d'attente *while* était à éviter, car elles bloquent l'exécution pendant un certain temps, durant lequel une alerte peut passer inaperçue.

Pour résoudre ce problème, le code a été conçu avec une approche non-bloquante basée sur des **Machines à États Finis (FSM)**. La boucle principale (`loop()`) tourne en permanence au lieu de bloquer le programme pour attendre un événement. La fonction `millis()` qui compte le temps depuis le démarrage est utilisée pour vérifier si un délai nécessaire est écoulé. Au cas contraire, le système passe immédiatement aux autres tâches sans attendre.

Cette architecture est séparée en deux machines à états :

— `Network_FSM` : Gère les différentes étapes de connexion réseau Wi-Fi et cellulaire.

— `Sensors_FSM` : Gère la lecture des capteurs et la publication des mesures et alertes.

Grâce à cette approche, le système vérifie toujours l'état des capteurs même lors de basculement de connexion.

2.3.2 Modélisation FSM de la résilience réseau

La machine à états `Network_FSM`, dont le diagramme est représenté sur la Figure 2.5, gère le basculement entre la connexion Wi-Fi et cellulaire en cas de coupure du Wi-Fi, afin d'assurer la continuité de la transmission des données par MQTT.

— **STATE_INIT_WIFI :** Par défaut, le système commence par tenter de se connecter au réseau Wi-Fi.

— **STATE_RUNNING_WIFI :** C'est l'état de fonctionnement normal. Le système maintient la connexion Wi-Fi et gère l'envoi des données vers le broker MQTT. Si la connexion Wi-Fi est perdue pendant plus de 10 secondes, qui est le délai de tolérance pour éviter de réagir à chaque micro-coupure, le système enclenche la procédure de basculement vers le réseau cellulaire.

— **STATE_START_CELLULAR :** Cet état initie l'allumage physique du modem SIM800L en pilotant ses broches d'alimentation (`MODEM_POWER_ON`) et de réinitialisation (`MODEM_RST`).

— **STATE_WAITING_MODEM_BOOT :** Le système attend 4s pour laisser le temps au matériel du modem de s'initialiser. Grâce à l'architecture non-bloquante, cette attente n'interrompt pas la surveillance continue des capteurs.

— **STATE_CONNECTING_GPRS :** Le microcontrôleur communique avec le modem pour s'attacher au réseau GSM, se connecter au GPRS et envoyer un SMS d'alerte notifiant la coupure du Wi-Fi. En cas d'échec de la connexion, l'automate retourne à l'état précédent pour recommencer la procédure d'initialisation.

— **STATE_RUNNING_CELLULAR :** Le système est connecté via le réseau cellulaire et assure la continuité de la transmission MQTT. Dans cet état, une vérification est effectuée toutes les 15s pour tester la disponibilité du Wi-Fi. Si le réseau Wi-Fi est de nouveau accessible, le modem SIM800L est mis hors tension pour limiter la consommation énergétique et le système retourne à l'état `STATE_INIT_WIFI`.

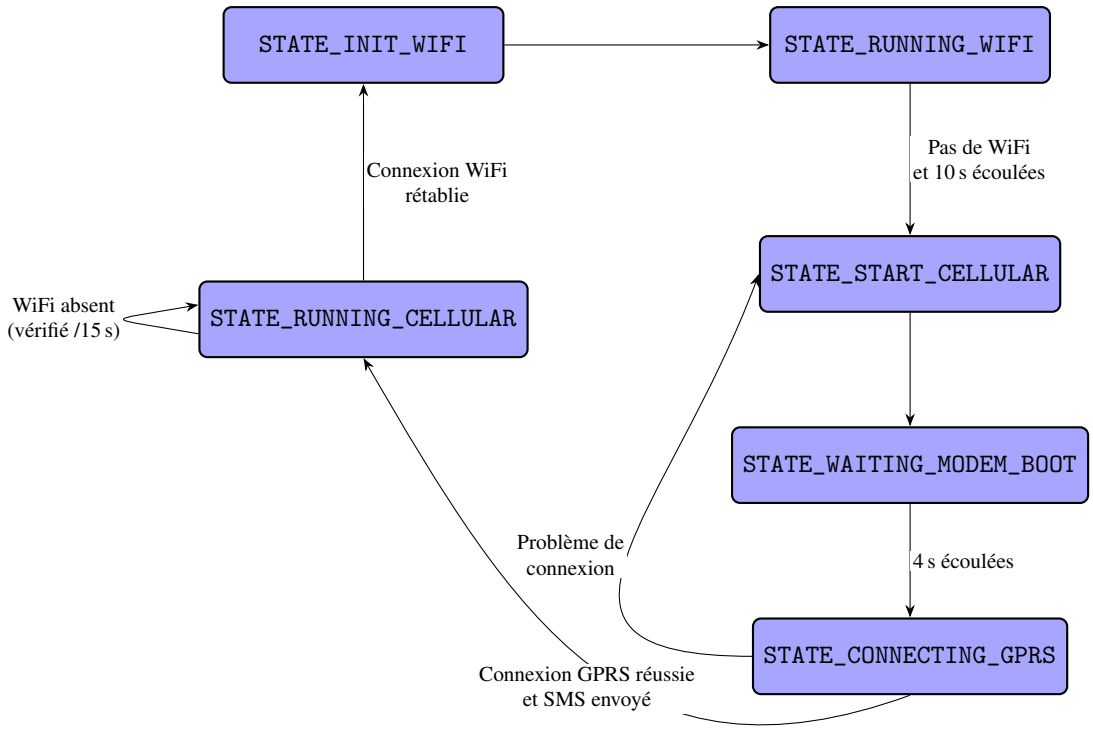


FIGURE 2.5 – Diagramme d'états de l'FSM de résilience réseau

2.3.3 Modélisation FSM de l'acquisition et de la gestion des alertes

La machine à états `Sensors_FSM`, dont le diagramme est illustré par la Figure 2.6, gère la lecture des différents capteurs et la logique de publication MQTT.

— **READ_AND_VERIFY :** C'est l'état d'écoute principal. Le système y effectue des lectures à deux vitesses : une lecture rapide toutes les 200 ms pour les événements urgents (gaz, flamme, porte, inondation) et une lecture toutes les 2 secondes pour les données environnementales et électriques. Une comparaison binaire avec l'opération `XOR` est réalisée entre les alertes actives et celles déjà signalées. Si une différence est détectée, le système bascule immédiatement vers les états de gestion d'alerte.

— **SEND_ALERT :** Déclenché lors de l'apparition d'une nouvelle situation anormale, cet état se charge de publier un message d'urgence sur le topic MQTT dédié. Dans le cas spécifique d'une coupure de courant, il enclenche également l'envoi d'un SMS via le modem. Une fois l'alerte envoyée et mémorisée, l'automate retourne à la surveillance.

— **CLEAR_ALERT :** Activé lorsqu'un capteur précédemment en alerte retourne à son état normal, le système publie alors un message de fin d'alerte. Les variables de suivi sont réinitialisées avant le retour à l'état de vérification.

— **SEND_ROUTINE :** En l'absence de toute nouvelle alerte ou de fin d'alerte, cet état est déclenché toutes les 5 secondes. Il publie les mesures régulières, puis retourne instantanément à l'état `READ_AND_VERIFY` pour commencer un nouveau cycle.

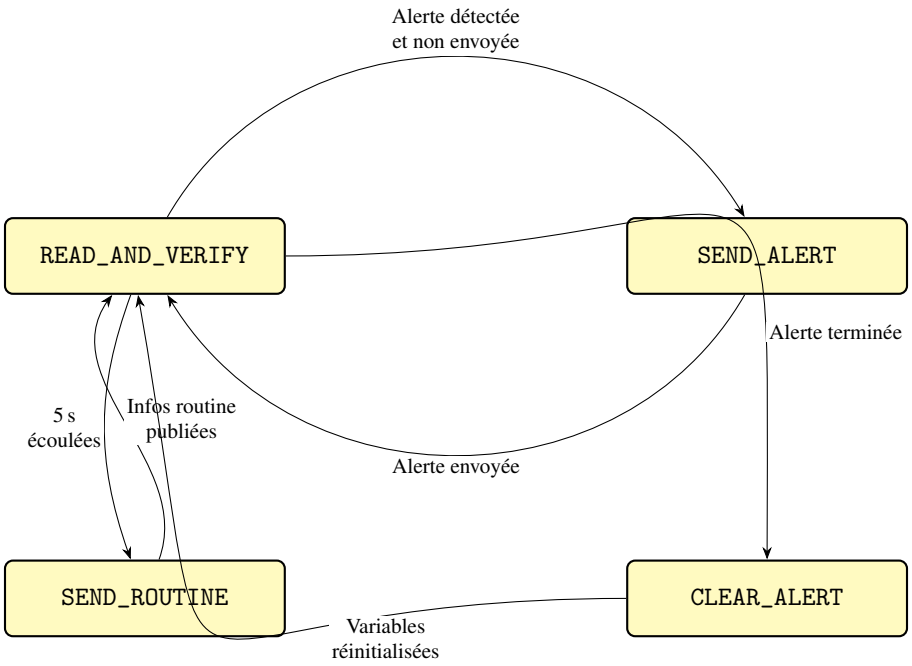


FIGURE 2.6 – Diagramme d'états de l'automate d'acquisition et de gestion des alertes

2.4 Développement et Programmation

Cette section détaille l'implémentation logicielle de chaque fonction clé du système, de la lecture des capteurs jusqu'à la transmission des alertes et des SMS d'urgence.
Le programme est organisé en plusieurs blocs fonctionnels indépendants appelés par la boucle principale.

2.4.1 Lecture des capteurs

La lecture des données de capteurs est gérée par la fonction `read_sensors()` dont le code source détaillé est disponible en Annexe A.1. Afin de garantir une réactivité maximale face aux dangers, la lecture a été divisée en deux vitesses grâce à la fonction `millis()`. Les capteurs environnementaux et électriques sont vérifiés toutes les 2 secondes car ils ont une inertie physique qui rend inutile une lecture très rapide. Par contre, les capteurs de sécurité sont vérifiés toutes les 200 ms pour garantir un déclenchement quasi instantané des alertes.

2.4.2 Vérification des alertes

Une fois les données acquises, la fonction `verifyAlerts()` détaillée dans l'annexe A.2 se charge de les comparer à des seuils de sécurité prédéfinis dans le code. L'état de toutes les alertes est stocké dans un entier de 16 bits. Chaque bit de cet entier représente un type d'alerte précis qui est actif à 1 et inactif à 0. Une opération de décalage binaire permet de mettre à 1 le bit correspondant à une alerte. L'accumulation par OU binaire (|=) permet de forcer le bit visé à 1 sans changer la valeur des autres bits déjà enregistrés. Cette méthode permet de représenter

tout l'état du système comme un seul mot binaire, ce qui simplifie la détection des changements d'état dans la machine à états, grâce à une comparaison binaire avec l'opération XOR réalisée entre les alertes actives et celles déjà signalées.

2.4.3 Publication MQTT : Mesures et Alertes

La communication avec le serveur est assurée par le protocole MQTT. Le système sépare l'envoi des mesures de routine et l'envoi des messages d'urgence. Pour les messages de routine, la fonction `publish_infos()`, détaillée en annexe A.3, formate les données sous forme d'une chaîne JSON. Ce format JSON est un standard textuel indépendant du langage de programmation. Il repose sur des paires clé-valeur organisées sous forme d'objets ou de listes. Il est léger, facile à intégrer aux outils de visualisation et élimine le risque de confondre l'ordre des données.

Pour les cas d'urgence, la fonction `publish_alerts()` , détaillée en annexe A.4, envoie un message sur un topic dédié aux alertes. Elle prend en paramètre le type d'alerte et un booléen qui indique s'il s'agit d'un déclenchement ou d'un retour à la normale.

Sur le plan de la sécurité, le prototype a été testé sur un broker MQTT public à des fins de développement.

2.4.4 Basculement réseau automatisé

La résilience de la communication repose sur la machine à états `Network_FSM` dont un extrait se trouve en Annexe A.5. Si la connexion Wi-Fi échoue, le système n'est pas bloqué. Au-delà de 10s de perte de connexion WiFi, il déclenche l'allumage du modem SIM800L. Une fois le modem allumé, la librairie MQTT passe automatiquement sur le client GPRS. En parallèle, un test est effectué toutes les 15s pour revenir sur le Wi-Fi dès que possible.

2.4.5 Transmission des SMS d'urgence

En cas de coupure de courant secteur, pour garantir la réception de cette alerte critique, la fonction `sendUrgentSMS()` dont le code figure sur l'Annexe A.6, pilote directement le module cellulaire. Cette fonction intègre une gestion intelligente de l'énergie. Si le modem GSM est déjà allumé, le message est envoyé instantanément. Si le modem était éteint, ce qui est le fonctionnement nominal sous Wi-Fi, la fonction l'allume, envoie le message, puis coupe l'alimentation du module afin de préserver l'autonomie restante de la carte.

2.5 Intégration d'une Interface Locale

Bien que le système soit conçu pour transmettre les données à distance, il s'est avéré utile d'ajouter une Interface Homme-Machine locale. L'ajout d'un écran permet à un opérateur présent sur place de consulter instantanément les mesures environnementales et électriques, ainsi que l'état des alertes.

2.5.1 Câblage et intégration matérielle

L'écran choisi est un module OLED SH1106 de résolution de 128×64 pixels. Son intégration matérielle a été effectuée via I2C. L'écran a été connecté aux broches `GPIO 21` (SDA) et `GPIO 22` (SCL) de l'ESP32 et alimenté en 3,3V [16]. Le schéma électrique complet, avec l'intégration de cet afficheur, est présenté à la Figure 2.7.

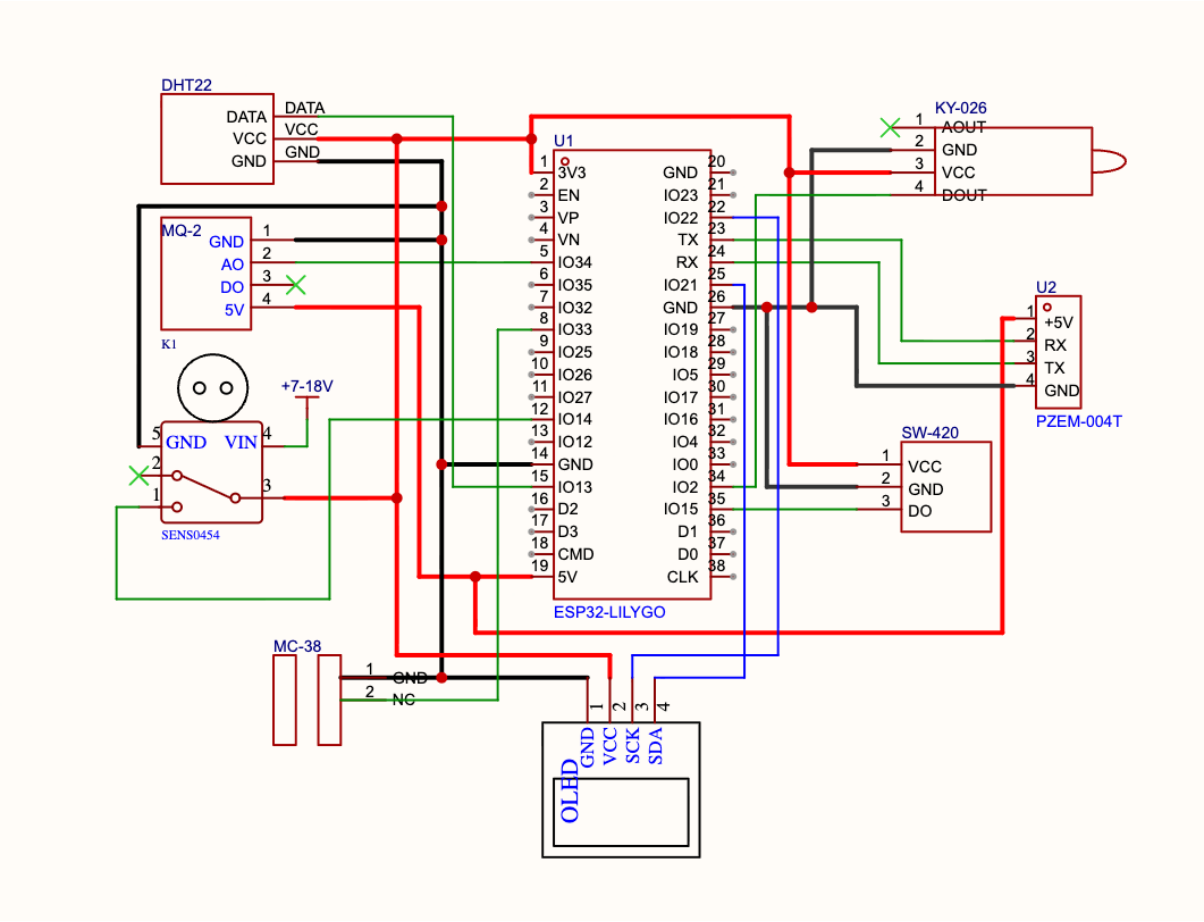


FIGURE 2.7 – Schéma de câblage incluant l'écran OLED

2.5.2 Gestion de l'affichage

Le principal défi lors de l'utilisation de l'afficheur de 128×64 pixels a été la gestion de sa surface d'affichage limitée. Il était impossible d'afficher simultanément toutes les mesures ainsi que les messages d'alerte. L'interface a donc été conçue de manière iconographique :

- **Télémétrie :** Les deux premières lignes de l'écran sont réservées à l'affichage des mesures de tension, courant, facteur de puissance, température, et humidité.
- **État du réseau :** Une icône au coin supérieur droit indique l'état actuel de la connectivité. Le système vérifie l'état de la machine `Network_FSM`, et si la connexion Wi-Fi est active, l'icône Wi-Fi s'affiche. En cas de basculement, elle est remplacée par l'icône du réseau cellulaire.
- **Matrice d'alertes dynamiques :** La moitié inférieure de l'écran est réservée aux alertes. Au lieu d'afficher du texte, des icônes spécifiques (12×12 pixels) ont été dessinées et encodées en mémoire. Ces icônes n'apparaissent que si l'alerte correspondante est active.

La Figure 2.8 présente une simulation de l'interface avec toutes les icônes d'alerte volontairement activées pour illustrer la disposition spatiale.

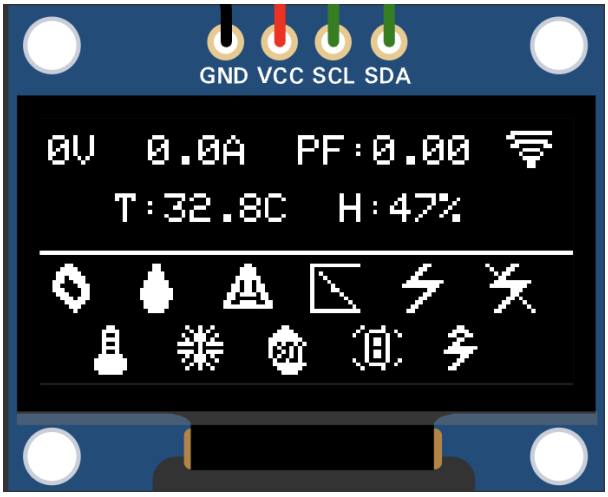


FIGURE 2.8 – Simulation de l'affichage OLED avec les icônes d'alerte actives

2.5.3 Intégration logicielle et gestion de l'affichage

L'intégration de l'écran repose sur les bibliothèques Adafruit_GFX et Adafruit_SH110X. La fonction d'affichage *updateDisplay()* est appelée directement dans la boucle principale *loop()*, indépendamment des deux machines à états Network_FSM et Sensors_FSM.
Elle intervient après leur exécution et applique un rafraîchissement non bloquant toutes les 200 ms grâce à la fonction *millis()*.
Cette fonction assure la mise à jour de l'interface en quelques étapes simples :

- **Télémétrie :** Centrage et affichage des mesures électriques et environnementales sur les premières lignes.
- **Statut réseau :** Sélection dynamique de l'icône Wi-Fi ou cellulaire en lisant l'état courant de *currentState*.
- **Alertes :** Lecture du masque binaire *activeAlertBits* pour dessiner uniquement les icônes des alertes actives, stockées en mémoire Flash.

Un extrait de la fonction *updateDisplay()* est fourni en Annexe A.7.

2.6 Tests et Résultats

Afin de valider le bon fonctionnement global du système, une série de tests expérimentaux a été menée. Ces essais couvrent l'initialisation du système, la mesure de la télémétrie, la réactivité des capteurs d'alerte, la transmission par SMS et MQTT, le basculement réseau ainsi que l'affichage sur l'écran OLED.

2.6.1 Initialisation et connectivité Wi-Fi

Dès la mise sous tension de la carte ESP32, la machine à états du réseau (`Network_FSM`) initie la connexion au point d'accès Wi-Fi configuré. Comme le montre la Figure 2.9, la carte

obtient directement une adresse IP dynamique (`192.168.43.218`) et s'enregistre auprès du réseau.

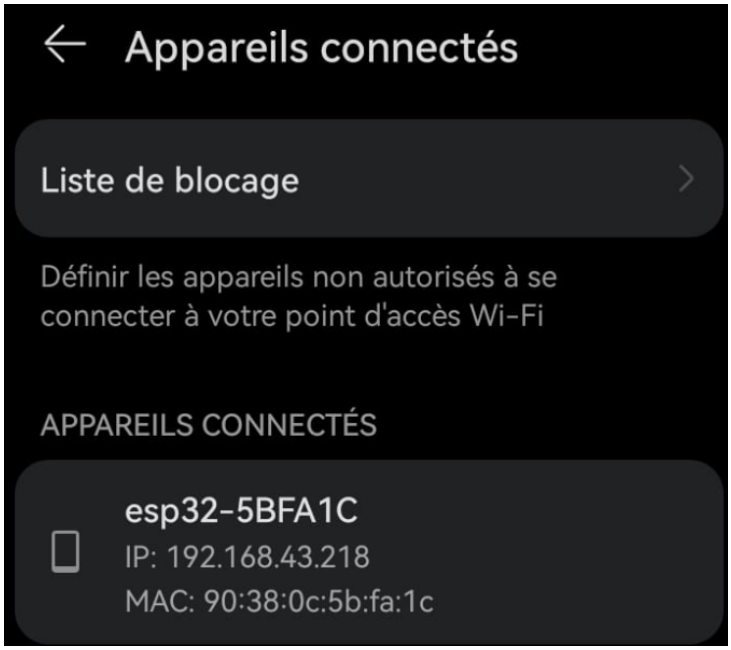


FIGURE 2.9 – Connexion établie au réseau Wi-Fi

2.6.2 Supervision électrique et alertes SMS

Un premier test a été effectué pour vérifier le comportement du système face à l'alimentation secteur :

— **Absence de tension secteur :** Lorsque la charge, une lampe, est débranchée, le capteur PZEM004T renvoie des valeurs nulles ($V = 0$V, $A = 0$A, $PF = 0.00$). Après détection de l'anomalie, la FSM des capteurs génère l'envoi d'un SMS d'urgence à l'opérateur (Figure 2.10) et publie les mesures sur le topic MQTT `telemetry/routine` (Figure 2.11).

— **Rétablissement du courant :** Lors du rebranchement de la charge, les mesures deviennent non nulles (Figure 2.12). Le système publie automatiquement un message de fin d'alerte sur le topic `alerts/critical` (Figure 2.13).

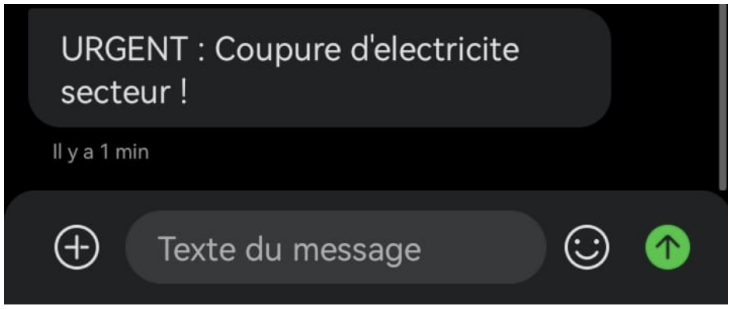


FIGURE 2.10 – SMS d'alerte reçu lors de la coupure de courant secteur

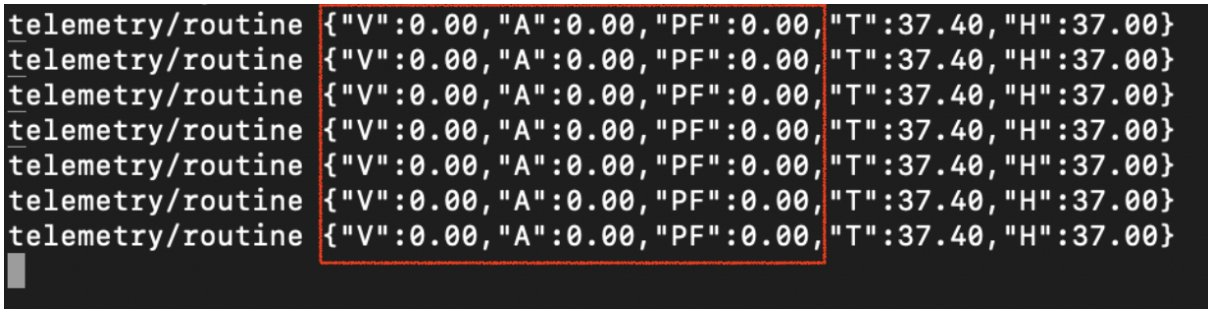


FIGURE 2.11 – Mesures publiées lors de la coupure de courant

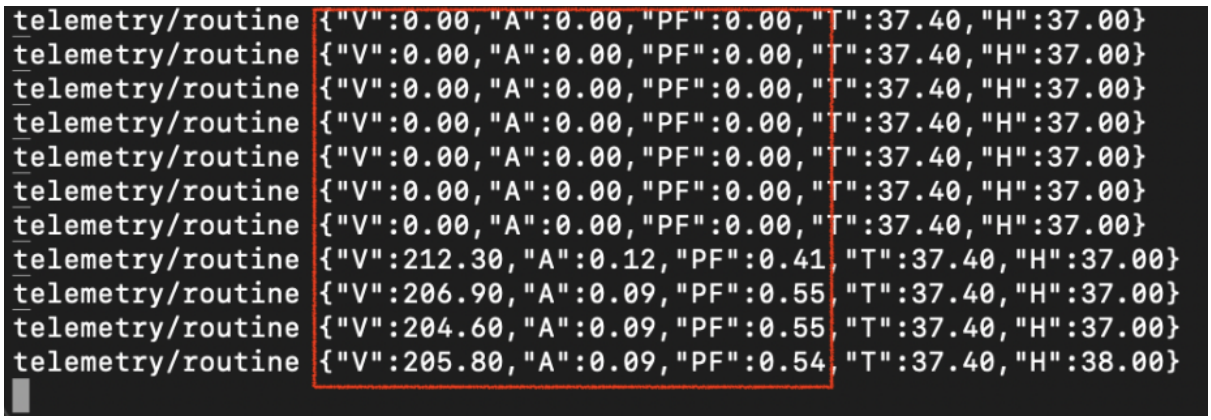


FIGURE 2.12 – Mesures publiées après le rebranchement de la charge

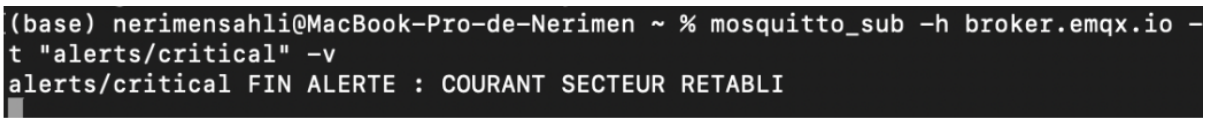


FIGURE 2.13 – Message de fin d'alerte coupure de courant

2.6.3 Validation des capteurs environnementaux et de sécurité

Chaque capteur a été testé individuellement afin de vérifier la cohérence des alertes émises et leur retour à la normale :

- **Fuite d'eau :** La simulation d'une inondation via un bouton-poussoir déclenche instantanément le message `"INONDATION / EAU"` sur le broker MQTT. Relâcher le bouton publie `"FIN ALERTE PLUS D'EAU"`.
- **Vibrations :** La simulation d'une vibration via un bouton-poussoir génère l'alerte `"VIBRATION ANORMALE"`, suivie du retour à la normale `"FIN ALERTE VIBRATION CORRIGEE"`.
- **Surchauffe thermique :** En chauffant le capteur DHT11 au-delà du seuil (40°C en cas de test), le message `"SURCHAUFFE TEMPERATURE"` est émis. Dès que la température repasse en dessous du seuil, un message de fin d'alerte est envoyé. Les variations de la température durant ce test sont visibles sur la Figure 2.15.
- **Humidité critique :** En exposant le capteur DHT11 à un taux d'humidité dépassant le seuil configuré (80%), la mesure augmente déclenchant immédiatement l'émission du

message `"HUMIDITE CRITIQUE"`. Dès que l'humidité repasse en dessous du seuil, un message de fin d'alerte est envoyé. L'évolution de la mesure d'humidité durant ce test est visible sur la Figure 2.16, et la capture du message d'alerte correspondant est présentée à la Figure 2.17.

— **Détection de flamme :** En approchant une flamme du capteur optique, l'alerte `"FLAMME DETECTEE"` apparaît, puis s'éteint dès l'extinction de la flamme.

— **Détection de gaz :** En exposant le capteur de gaz à une concentration dépassant le seuil configuré, le système déclenche immédiatement l'émission du message d'alerte `"FUITE DE GAZ"`. Lorsque le gaz se dissipe et que la valeur mesurée repasse sous le seuil, le système publie automatiquement le message de fin d'alerte `"FIN ALERTE : GAZ DISSIPE"`.

Les résultats des tests effectuées pour les cas de fuite d'eau, de vibration et de surchauffe sont illustrés sur la Figure 2.14, et ceux pour le cas de détection d'une flamme sont illustrés sur la Figure 2.18.

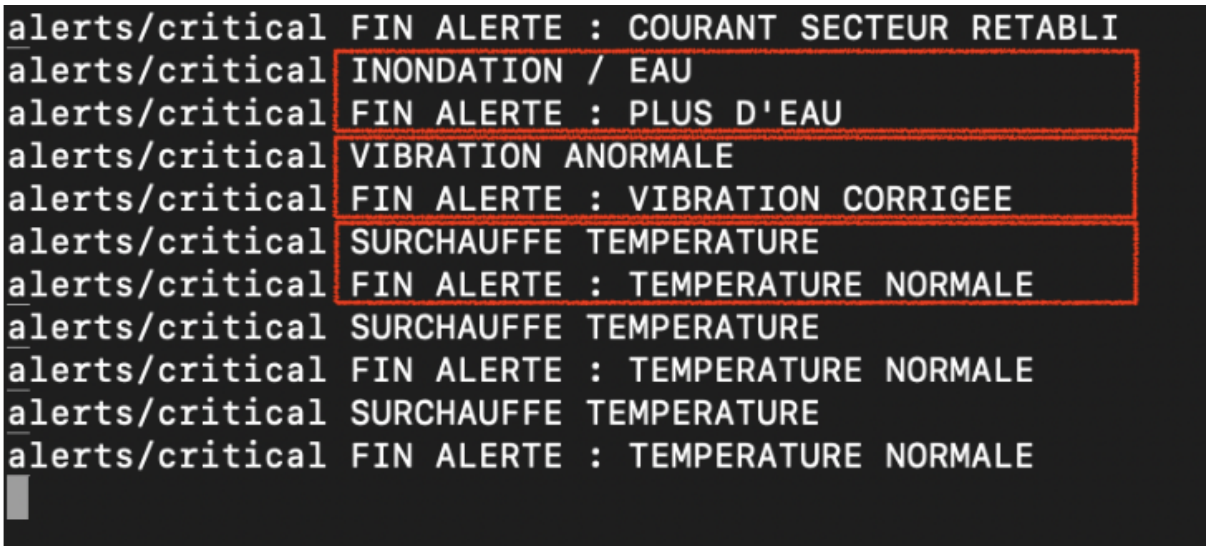


FIGURE 2.14 – Résultats des tests effectuées pour les cas de fuite d'eau, de vibration et de surchauffe

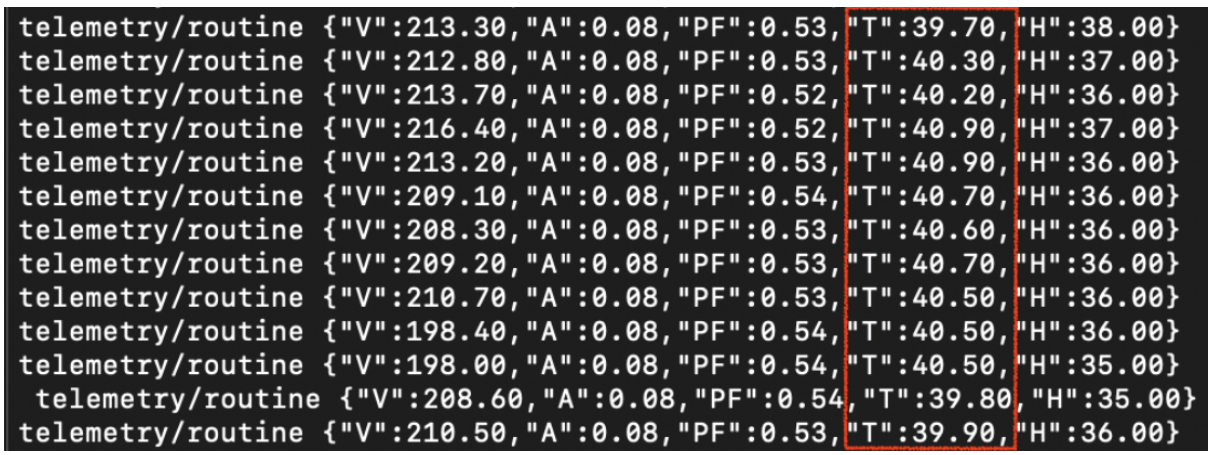


FIGURE 2.15 – Variation de la température autour du seuil prédéfini

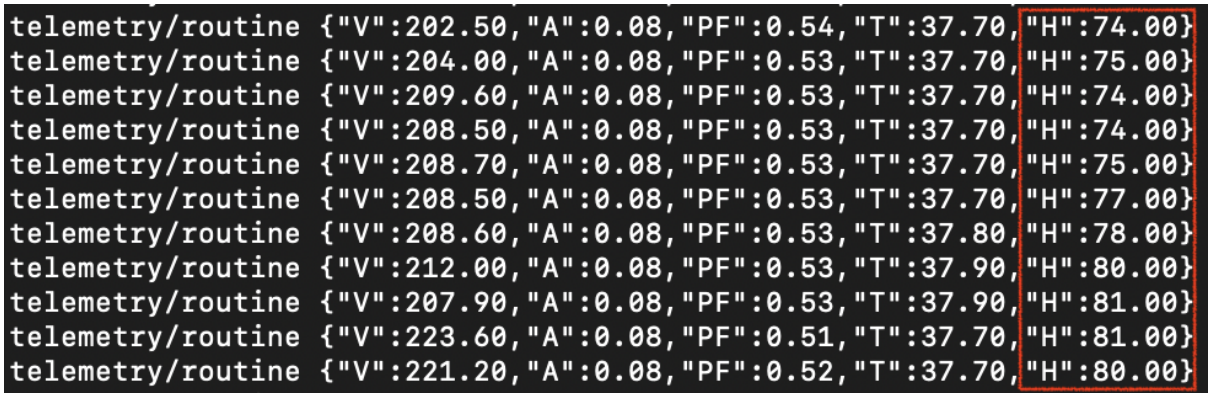


FIGURE 2.16 – Augmentation de l'humidité jusqu'au seuil prédéfini



FIGURE 2.17 – Publication de l'alerte en cas d'humidité élevée



FIGURE 2.18 – Résultats des tests effectuées pour le cas de détection d'une flamme

Il est à noter que les seuils utilisés lors des tests ont été fixés arbitrairement pour permettre leur déclenchement dans les conditions de réalisation du prototype, et devront être recalibrés selon les normes et les spécifications du site lors du déploiement réel.

2.6.4 Superposition et indépendance des alertes

Dans le but de vérifier que les alertes sont envoyées indépendamment l'une de l'autre, un scénario de pannes simultanées a été testé : La porte de l'armoire a été ouverte en séparant les aimants, générant l'alerte `"PORTE OUVERTE"`. Tout en maintenant la porte ouverte, une fuite d'eau a été simulée. Enfin, la porte de l'armoire a été refermée.

Les résultats sur la Figure 2.19 confirment que les deux alertes sont gérées indépendamment par le masque binaire `activeAlertBits`. L'apparition et la résolution de l'inondation se sont déroulées sans affecter l'état de l'alerte de porte ouverte, prouvant ainsi la robustesse de la FSM.

```
alerts/critical PORTE OUVERTE
alerts/critical INONDATION / EAU
alerts/critical FIN ALERTE : PLUS D'EAU
alerts/critical INONDATION / EAU
alerts/critical FIN ALERTE : PLUS D'EAU
alerts/critical FIN ALERTE : PORTE FERMEE
```

FIGURE 2.19 – Résultat du test avec un scénario de pannes simultanées

2.6.5 Mécanisme de basculement réseau

Pour tester la résilience des communications, le partage Wi-Fi a été déconnecté manuellement pendant le fonctionnement du système :

1. La rupture de liaison Wi-Fi est détectée par la `Network_FSM`.
2. Le module GSM SIM800 est alimenté et envoie un SMS de notification (Figure 2.20).
3. Le système établit la connexion cellulaire et reprend immédiatement la publication.
4. Dès la réapparition du signal Wi-Fi, la carte bascule à nouveau automatiquement sur le Wi-Fi et coupe la session GPRS.



FIGURE 2.20 – SMS d'alerte reçu lors de la coupure de WiFi

2.6.6 Validation de l'affichage sur l'écran OLED

Le rendu de l'interface graphique a été validé sur l'écran OLED. La Figure 2.21 présente une photo réelle de l'écran lors du fonctionnement du système.

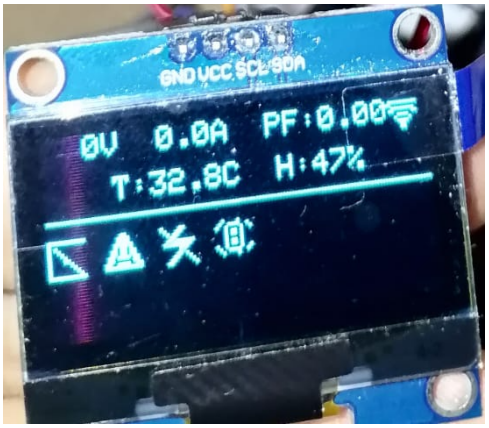


FIGURE 2.21 – L'écran OLED affichant la télémétrie, le statut réseau et des icônes d'alerte

L'essai sur le matériel réel confirme le bon fonctionnement de l'affichage :

— Les deux premières lignes affichent clairement les grandeurs physiques en temps réel (Tension, Courant, Facteur de puissance, Température, Humidité).

— L'icône en haut à droite s'adapte dynamiquement selon le mode de connexion actif (Wi-Fi ou GSM).

— Les icônes d'alerte apparaissent instantanément dès qu'une anomalie est détectée.

2.7 Analyse de l'Empreinte Mémoire

Afin de valider l'efficacité du système et d'anticiper sa stabilité en fonctionnement continu, une évaluation des ressources mémoire consommées a été effectuée lors de la compilation sous Arduino IDE.

— **Mémoire Flash (Programme) :** Le binaire compilé occupe **987 955 octets**, ce qui constitue **75%** de l'espace alloué qui est de 1 310 720 octets. Cela confirme que l'ensemble des bibliothèques et des machines à états finis tient confortablement dans le microcontrôleur. La réserve de 25% garantit la possibilité d'intégrer de futures fonctionnalités.

— **Mémoire SRAM (Variables dynamiques) :** Les variables globales et statiques n'occupent que **47 072 octets**, soit **14%** de la mémoire vive disponible qui est de 327 680 octets. Le maintien de **86% de SRAM libre** pour l'exécution dynamique prévient tout risque de saturation mémoire ou d'instabilité lors des reconnexions réseau et du traitement des alertes.

2.8 Bilan Financier et Estimation des Coûts

Afin d'évaluer l'intérêt économique du projet pour l'entreprise CCNS, une étude financière comparative a été réalisée. Celle-ci compare le coût de fabrication du prototype développé et les coûts des solutions industrielles équivalentes analysées lors de l'étude de marché.

2.8.1 Nomenclature des coûts du prototype

Le Tableau suivant 2.1 détaille le coût des composants matériels constituant le système.

TABLE 2.1 – Bilan financier de la réalisation du prototype

Composant	Référence / Modèle	Qté	Prix Est. (TND)
Microcontrôleur & Modem	LilyGO TTGO T-Call	1	85,00 DT
Mesureur Électrique	PZEM-004T + CT	1	45,00 DT
Capteur Température/Humidité	DHT22	1	16,00 DT
Détecteur de Gaz	MQ-135	1	12,00 DT
Capteur de Flamme	KY-026	1	8,00 DT
Capteur de Vibration	SW-420	1	6,00 DT
Capteur Ouverture Porte	MC-38	1	5,00 DT
Détecteur d'Inondation	SEN0454	1	25,00 DT
Afficheur Graphique Local	OLED SH1106 (128x64)	1	18,00 DT
Autres	Abaisseur de niveau, plaque d'essai	1	15,00 DT
Coût Total Matériel			**235,00 DT**

2.8.2 Analyse comparative avec les solutions du marché

Les solutions de supervision pour Data Centers dans le marché présentent des coûts d'investissement élevés. Le Tableau 2.2 compare le prototype développé aux trois principales solutions du marché analysées précédemment.

TABLE 2.2 – Comparaison financière entre le prototype et les solutions du marché

Solution	Constructeur	Prix (€ / $)	Équiv. (TND)
NetBotz 355/455	Schneider Electric / APC	≈ 1 200 à 2 500 €	4 000 à 8 300 DT
Room Alert 12E	AVTECH Software	≈ 1 200 $	≈ 3 900 DT
Smart Controller	Raritan (Legrand)	≈ 1 000 $	≈ 3 300 DT
Prototype CCNS	**Projet sur mesure**	≈ 70 €	**235 DT**

2.8.3 Synthèse et Gain Économique

L'analyse des coûts met en évidence des avantages majeurs :

— **Réduction drastique du coût matériel :** Le prototype proposé coûte environ **15 à 30 fois moins cher** que les boîtiers propriétaires du marché.

— **Absence de licence :** Les solutions propriétaires imposent souvent des abonnements logiciels mensuels ou annuels pour l'utilisation des passerelles Cloud et l'envoi de SMS via plateforme tierce alors que le système conçu repose sur une pile Open Source et une carte SIM standard.

Il est toutefois à noter, pour nuancer cette comparaison, que ces solutions commerciales incluent des éléments non couverts par le prototype actuel, comme un boîtier certifié pour environnement industriel, un support technique dédié, une plateforme logicielle éprouvée et une garantie constructeur.

2.9 Conclusion et Perspectives d'Évolution

Ce travail de conception et de réalisation a permis de valider le fonctionnement complet du nœud de capteurs intelligent qui répond fidèlement aux exigences fonctionnelles du cahier des charges :

— Mesure et transmission des grandeurs électriques et environnementales via MQTT.

— Détection indépendante de multiples alertes physiques (inondation, gaz, flamme, porte, vibration, surchauffe, variations de tension/courant) sans délai perceptible.

— Maintien de la connectivité grâce au basculement automatique entre le réseau Wi-Fi et le réseau cellulaire.

— Notification par SMS lors d'une coupure de courant secteur ou du signal Wi-Fi.

— Affichage local sur écran OLED.

Bien que fonctionnel, le prototype présente certaines limites qu'il convient de souligner : l'utilisation de composants de substitution comme le DHT11 et les boutons-poussoirs en lieu

de certains capteurs, des seuils d'alerte fixés arbitrairement pour les besoins des tests, l'absence de mesure précise des temps de latence et l'usage d'un broker MQTT de test non sécurisé.

Plusieurs perspectives d'évolution peuvent être envisagées pour faire de ce prototype un produit industriel :

— Conception d'un circuit imprimé (PCB) qui remplace le câblage sur plaque d'essai pour éliminer les faux contacts.

— Intégration d'une batterie de secours rechargeable pour alimenter le système lors d'une coupure électrique.

— Concevoir un boîtier sur mesure conforme aux normes industrielles pour protéger le système en environnement sévère.

Conclusion générale

Ce stage d'ingénieur, effectué au sein de l'entreprise CCNS, avait pour objectif la conception et la réalisation d'un nœud de capteurs IoT dédié à la surveillance des Data Centers et salles serveurs.

Tout au long de ce projet a été menée une démarche d'ingénierie qui couvre de l'analyse des besoins et l'étude bibliographique jusqu'à la validation expérimentale d'un prototype physique. Un accent particulier a été mis sur la phase de validation à travers la simulation de multiples scénarios et de problèmes potentiels. La combinaison du microcontrôleur ESP32, du modem SIM800L et du protocole MQTT a permis de réaliser une passerelle physique-virtuelle assez performante, économique et réactive.

L'apport majeur de ce travail réside dans l'utilisation d'une architecture logicielle s'appuyant sur des machines à états finis non-bloquantes. Cette approche garantit la détection d'alertes de façon continue grâce au basculement automatique vers le réseau cellulaire et à l'envoi des SMS d'urgence en cas de défaillance de l'infrastructure réseau ou d'une coupure d'électricité.

Sur le plan personnel et professionnel, ce stage a constitué pour moi une expérience enrichissante. Il m'a permis d'approfondir mes compétences en systèmes embarqués, en programmation C/C++, en protocoles de communication IoT et en intégration matérielle/logicielle, tout en renforçant ma capacité de résolution de problèmes. Il m'a également offert l'opportunité de m'adapter aux exigences de la vie professionnelle et d'apporter une contribution réelle à l'entreprise CCNS.

Bibliographie et Webographie

[1] **CCNS (Company of Computing and Network Services)**, *Site institutionnel et présentation des services*.
URL : https://www.ccn-services.com/

[2] **Motilde**, *Gestion des Infrastructures de Data Center (DCIM)*.
URL : https://motilde.com/data-center-infrastructure-management-dcim/

[3] **Schneider Electric / APC**, *NetBotz Environmental Security and Data Center Monitoring Solutions*.
URL : https://www.se.com/us/en/product-range/61830-netbotz/

[4] **AVTECH Software Inc.**, *Room Alert Environment Monitoring Solutions for Data Centers*.
URL : https://fr.avtech.com/

[5] **Raritan (Legrand)**, *Intelligent Sensors and Smart Rack Controller (SRC) Specifications*.
URL : https://www.raritan.com/eu/fr/products/power/rack-management-intelligent-sensor

[6] **Come-Star**, *MQTT vs HTTP : Key Differences, Performance in IoT and Selection Guide*.
URL : https://www.come-star.com/fr/blog/mqtt-vs-http-key-differences-performance-in-iot-and-how-to-choose/

[7] **EMQX**, *ESP32 Connects to the Free Public MQTT Broker Guide*.
URL : https://www.emqx.com/en/blog/esp32-connects-to-the-free-public-mqtt-broker

[8] **LilyGO**, *TTGO T-Call ESP32 with SIM800L GSM/GPRS Module Datasheet & Pinout Manual*.
URL : https://manuals.plus/ae/1005008990936049

[9] **Peacefair Electronics**, *PZEM-004T V3.0 AC Multifunction Power Meter Datasheet and Communication Protocol*.
URL : https://innovatorsguru.com/wp-content/uploads/2019/06/PZEM-004T-V3.0-Datasheet-User-Manual.pdf

[10] **DFRobot**, *SEN0454 Immersion Water Leak Detector Technical Documentation*.
URL : https://www.mouser.com/pdfDocs/Product-Overview-DFRobot-SEN0454.pdf

[11] **Tuni-Smart Innovation**, *Capteur de Température et d'Humidité DHT22 / AM2302 Specifications*.
URL : https://tuni-smart-innovation.com/products/dht22-capteur-de-temperature-et-dhumidite

[12] **Winsen Electronics**, *MQ-135 Semiconductor Sensor for Air Quality and Hazardous Gases Datasheet*.

URL : https://ja-bots.com/wp-content/uploads/2025/04/MQ-135-datasheet.pdf

[13] **Arduino Modules**, *KY-026 Optical Flame Sensor Module Datasheet*.
URL : https://arduinomodules.info/ky-026-flame-sensor-module/

[14] **Components101**, *MC-38 Magnetic Door Switch Sensor Pinout, Features & Working Principle*.
URL : https://components101.com/sensors/mc38-magnetic-switch-sensor-pinout-features-datasheet-working-alternative-application

[15] **HandsOn Technology**, *SW-420 Motion and Vibration Sensor Module Specification*.
URL : https://handsontec.com/dataspecs/sensor/Vibration%20Sensor.pdf

[16] **Sino Wealth / Velleman**, *SH1106 128x64 Dot Matrix OLED Driver/Controller Datasheet*.
URL : https://cdn.velleman.eu/downloads/29/infosheets/sh1106_datasheet.pdf

Annexe A

Extraits du code

A.1 Code de lecture des capteurs

```
void read_sensors(unsigned long now){
  if ((now - lastSlowReading) >= 2000){
    float t = dht.readTemperature();
    float h = dht.readHumidity();
    if (!isnan(t)) temperature = t;
    if (!isnan(h)) humidity = h;
    if (pzem != nullptr) {
      float v = pzem->voltage();
      float a = pzem->current();
      float pf = pzem->pf();
      bool voltageValid = !isnan(v) && v > 100.0 && v < 265.0;
      bool currentValid  = !isnan(a) && a >= 0.0 && a < 30.0;
      if (voltageValid) {
        voltage = v;
        firstValidReadingDone = true;
        invalidVoltageCount = 0;
      } else {
        voltage = 0.0;
        invalidVoltageCount++;
      }
      current = (voltageValid && currentValid) ? a : 0.0;
      powerFactor = (voltageValid && !isnan(pf)) ? pf : 0.0;
    }
    lastSlowReading = now;
  }
  if ((now - lastFastReading) >= 200){
    WaterDetected = digitalRead(WaterLeakSensor);
    VibrationDetected = digitalRead(VibrationSensor);
    GazDetected = analogRead(GAZPIN);
    Serial.println(analogRead(34));
    FlameDetected = digitalRead(FLAMEPIN);
    DoorOpen = digitalRead(DOORPIN);
    lastFastReading = now;
  }
}
```

A.2 Code de vérification des alertes

```
uint16_t verifyAlerts() {
  uint16_t bits = 0;
  if (FlameDetected == HIGH)       bits |= (1 << FlameFound);
  if (GazDetected > gazThreshold) bits |= (1 << GazFound);
  if (WaterDetected == HIGH)       bits |= (1 << WaterFound);
  if (temperature > tempLimitMAX) bits |= (1 << TemperatureHigh);
  if (temperature < tempLimitMIN) bits |= (1 << TemperatureLow);
  if (humidity > humidityLimit)    bits |= (1 << HumidityHigh);
  if (VibrationDetected == HIGH)  bits |= (1 << VibrationFound);
  if (DoorOpen == HIGH)            bits |= (1 << DoorOpened);
  if (pzem != nullptr) {
  if (firstValidReadingDone) {
    if (voltage > voltageLimit) bits |= (1 << VoltageHigh);
    if (current > currentLimit) bits |= (1 << CurrentHigh);
  }
  if (invalidVoltageCount >= CURRENT_CUT_THRESHOLD) bits |= (1 <<
      CurrentCut);
  }
  return bits;
}
```

A.3 Code de publication des mesures

```
void publish_infos() {
  if (!client.connected()) return;
  String payload = "{\"V\":"  + String(voltage, 2) +
                   ",\"A\":"  + String(current, 2) +
                   ",\"PF\":" + String(powerFactor, 2) +
                   ",\"T\":"  + String(temperature, 2) +
                   ",\"H\":"  + String(humidity, 2) + "}";
  client.publish("CCNS2026/telemetry/routine", payload.c_str());
}
```

A.4 Code de publication des alertes

```
void publish_alerts(AlertType type, bool active){
  if (!client.connected()) return;
  String message = "";
  if (active) {
    switch (type) {
      case FlameFound:       message = "FLAMME DETECTEE"; break;
      case GazFound:         message = "FUITE DE GAZ"; break;
      case WaterFound:       message = "INONDATION / EAU"; break;
      case TemperatureHigh:  message = "SURCHAUFFE TEMPERATURE"; break;
      case TemperatureLow:   message = "TEMPERATURE TROP BASSE"; break;
      case HumidityHigh:     message = "HUMIDITE CRITIQUE"; break;
      case VibrationFound:   message = "VIBRATION ANORMALE"; break;
      case DoorOpened:       message = "PORTE OUVERTE"; break;
      case VoltageHigh:      message = "SURTENSION ELECTRIQUE"; break;
      case CurrentHigh:      message = "SURINTENSITE ELECTRIQUE"; break
          ;
      case CurrentCut:       message = "COUPURE DE COURANT SECTEUR";
         break;
      default:               message = "ALERTE INCONNUE"; break;
    }
  } else {
    switch (type) {
      case FlameFound:       message = "FIN ALERTE : FLAMME CORRIGEE";
         break;
      case GazFound:         message = "FIN ALERTE : GAZ DISSIPE";
         break;
      case WaterFound:       message = "FIN ALERTE : PLUS D'EAU"; break
          ;
      case TemperatureHigh:  message = "FIN ALERTE : TEMPERATURE
         NORMALE"; break;
      case TemperatureLow:   message = "FIN ALERTE : TEMPERATURE
         NORMALE"; break;
      case HumidityHigh:     message = "FIN ALERTE : HUMIDITE NORMALE";
          break;
      case VibrationFound:   message = "FIN ALERTE : VIBRATION CORRIGEE
         "; break;
      case DoorOpened:       message = "FIN ALERTE : PORTE FERMEE";
         break;
      case VoltageHigh:      message = "FIN ALERTE : TENSION NORMALE";
         break;
      case CurrentHigh:      message = "FIN ALERTE : COURANT NORMAL";
         break;
      case CurrentCut:    message = "FIN ALERTE : COURANT SECTEUR
         RETABLI"; break;
      default:               message = "FIN ALERTE : RETOUR A LA
         NORMALE"; break;
    }
  }
  client.publish("CCNS2026/alerts/critical", message.c_str());
}
```

A.5 Extrait de la FSM réseau

```
    case STATE_INIT_WIFI:
      // tentative de connexion au WiFi
    case STATE_RUNNING_WIFI:
       if (WiFi.status() == WL_CONNECTED) {
        // transmission via WiFi
      }
      else if (WiFi.status() != WL_CONNECTED && (now - stateTimer >
          10000)) {
        currentState = STATE_START_CELLULAR;
      }
      break;
    case STATE_START_CELLULAR:
      modemReady = false;
      // allumage du modem
      stateTimer = now;
      currentState = STATE_WAITING_MODEM_BOOT;
      break;
    case STATE_WAITING_MODEM_BOOT:
      if (now - stateTimer >= 4000) {
          modemSerial.begin(115200, SERIAL_8N1, MODEM_RX, MODEM_TX);
          currentState = STATE_CONNECTING_GPRS;
          stateTimer = now;
      }
      break;
    case STATE_CONNECTING_GPRS:
      // v rifier l'absence de probl mes de connexion
      modemReady = true;
      // envoi d'un SMS d'urgence
      if (!modem.gprsConnect(apn, gprsUser, gprsPass)) {
        currentState = STATE_START_CELLULAR;
        break;
      }
      client.setClient(bstGPRSClient);
      currentState = STATE_RUNNING_CELLULAR;
      break;
    case STATE_RUNNING_CELLULAR :
      if (!client.connected() && (now - lastMQTTRetry > 5000)) {
        // transmission via cellulaire
      }
      if (now - lastWiFiCheck > 15000) {
        lastWiFiCheck = now;
        if (WiFi.status() == WL_CONNECTED ) {
          //   teinte   du modem
          currentState = STATE_INIT_WIFI;
        }
      }
      break;
```

A.6 Fonction de l'envoi d'SMS

```
bool sendUrgentSMS(const char* message) {
  if (modemPoweredOn && !modemReady) {
    return false;
  }

  bool openedSpecificallyForSMS = false;

  if (!modemPoweredOn) {
    openedSpecificallyForSMS = true;

    pinMode(MODEM_PWKEY, OUTPUT);
    pinMode(MODEM_RST, OUTPUT);
    pinMode(MODEM_POWER_ON, OUTPUT);

    digitalWrite(MODEM_POWER_ON, HIGH);
    digitalWrite(MODEM_RST, HIGH);
    delay(100);
    digitalWrite(MODEM_RST, LOW);
    delay(100);
    digitalWrite(MODEM_RST, HIGH);

    digitalWrite(MODEM_PWKEY, LOW);
    delay(1000);
    digitalWrite(MODEM_PWKEY, HIGH);
    delay(4000);

    modemSerial.begin(115200, SERIAL_8N1, MODEM_RX, MODEM_TX);
    modemPoweredOn = true;

    if (!modem.init() || !modem.waitForNetwork(10000L)) {
      digitalWrite(MODEM_POWER_ON, LOW);
      modemPoweredOn = false;
      return false;
    }
  }

  bool sent = modem.sendSMS(target_phone_number, message);

  if (openedSpecificallyForSMS) {
    digitalWrite(MODEM_POWER_ON, LOW);
    modemPoweredOn = false;
    modemReady = false;
  }

  return sent;
}
```

A.7 Fonction d'affichage sur l'écran

```
void updateDisplay() {
  if (!oledReady) return;
  unsigned long now = millis();
  if (now - lastDisplayUpdate < 200) return;
  lastDisplayUpdate = now;

  display.clearDisplay();
  display.setTextColor(SH110X_WHITE);
  display.setTextSize(1);
  String line1 = String((int)voltage) + "V  " + String(current, 1) + "A
       PF:"
  + String(powerFactor, 2);
  drawCenteredText(line1, 0);

  String line2 = "T:"
  + String(temperature, 1) + "C  H:" + String((int)humidity) + "%";
  drawCenteredText(line2, 14);
  display.drawFastHLine(0, 26, 128, SH110X_WHITE);

  int x = 0;
  int y = 32;
  for (int i = 1; i < ALERT_TYPE_COUNT; i++) {
    if (activeAlertBits & (1 << i)) {
      display.drawBitmap(x, y, alertIcons[i], 12, 12, SH110X_WHITE);
      x += 18;
      if (x > 108) {
        x = 0;
        y += 16;
      }
    }
  }
  bool onWiFi = (currentState == STATE_INIT_WIFI) || (currentState
  == STATE_RUNNING_WIFI);
  display.drawBitmap(SCREEN_WIDTH - 14, 0, onWiFi ? icon_wifi :
  icon_cellular, 12, 12, SH110X_WHITE);
  display.display();
}
```